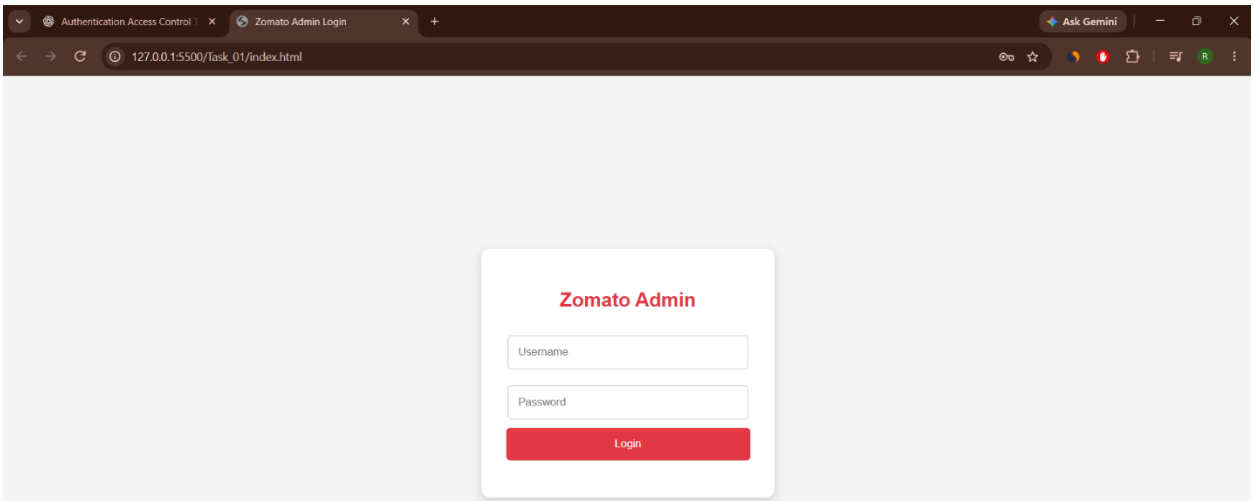
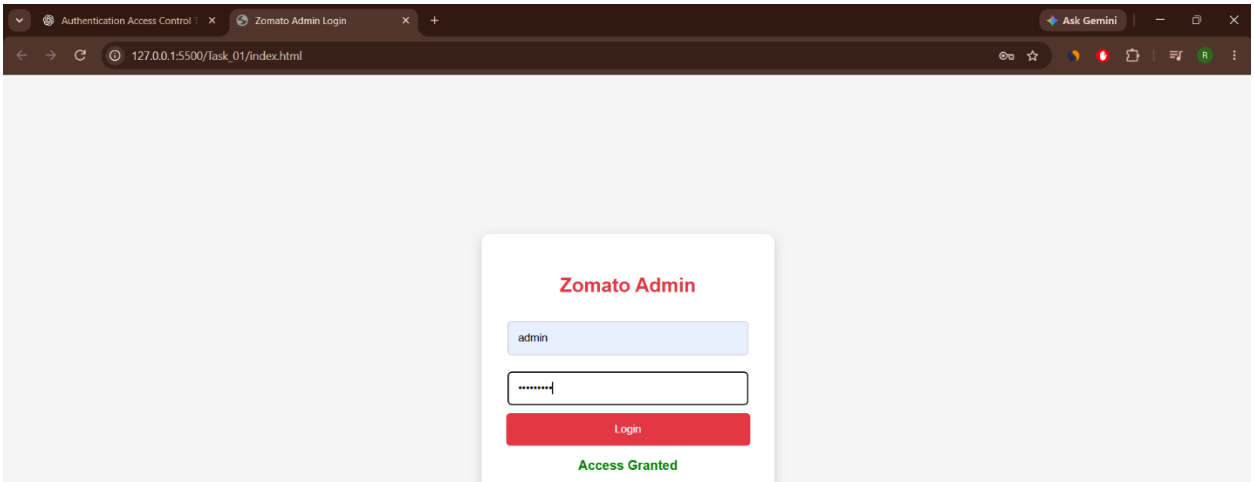
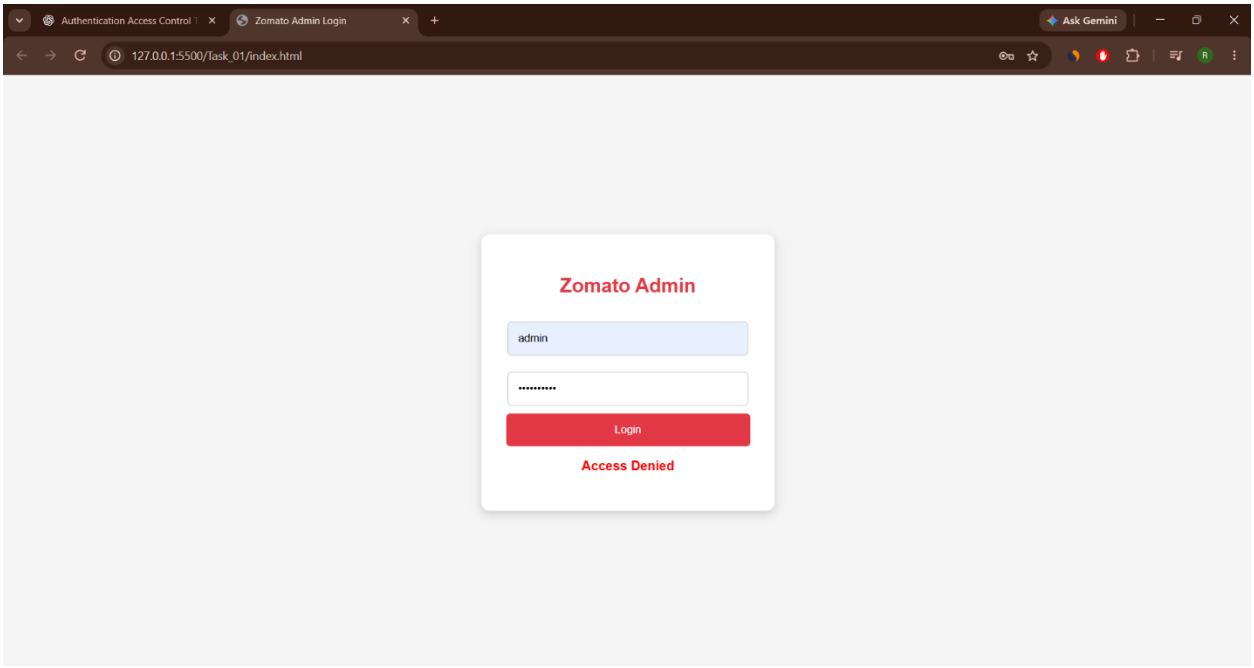
# Authentication and Access Control

**Task 1 : Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.**






To create a simple Zomato Admin Login Form using HTML, CSS and JavaScript.

**Description:**
The login form accepts a username and password. It checks the entered credentials against predefined admin credentials. If both credentials match, it displays "Access Granted"; otherwise, it displays "Access Denied".

**Admin Credentials:**

- Username: admin
- Password: zomato123

**Output:**

1. Successful Login – Access Granted.
2. Incorrect Login – Access Denied.

**Conclusion:**
Successfully created and tested a basic login authentication form using HTML, CSS and JavaScript.

**Task 2 : Write a Python function check_access(user_role, resource) that returns True if the user_role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant_owner', and resources like 'menu_edit', 'order_history', 'admin_dashboard'.**

To create a Python function that checks whether a user role is allowed to access a particular resource.

**Description:**
A Python dictionary is used to store roles and their permitted resources. The check_access() function checks the user's role and returns True if access is allowed; otherwise, it returns False.

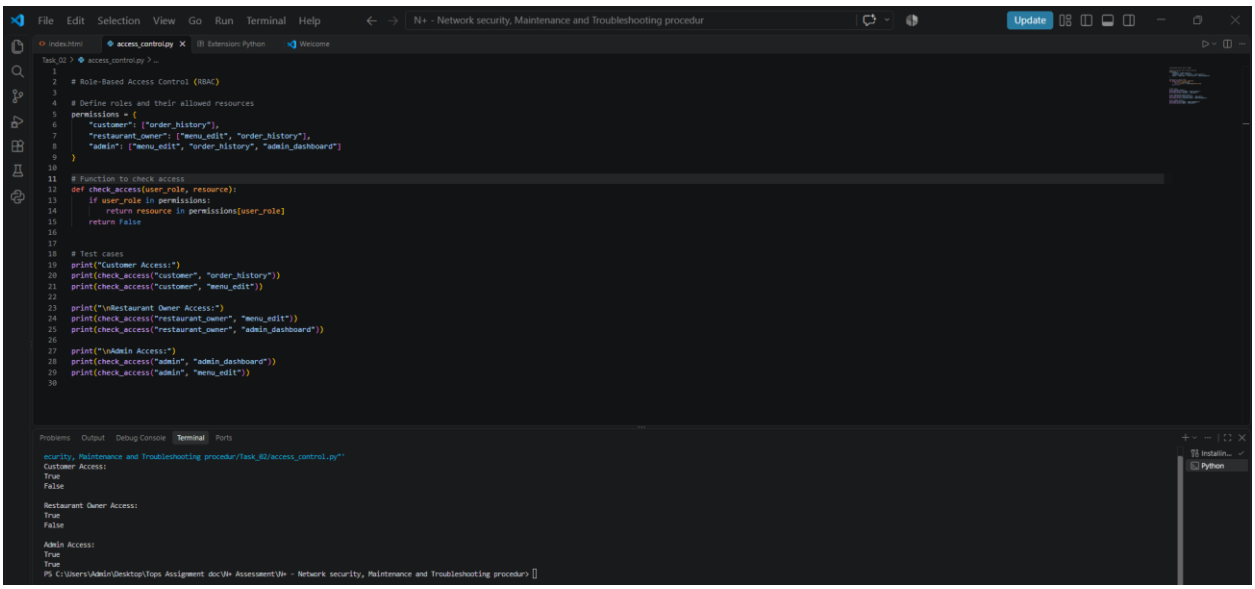
**Roles Used:**

- Customer
- Restaurant Owner
- Admin

**Resources Used:**

- menu_edit
- order_history
- admin_dashboard

**Output:**
The program successfully checks access permissions for different user roles and displays True or False based on the permissions.



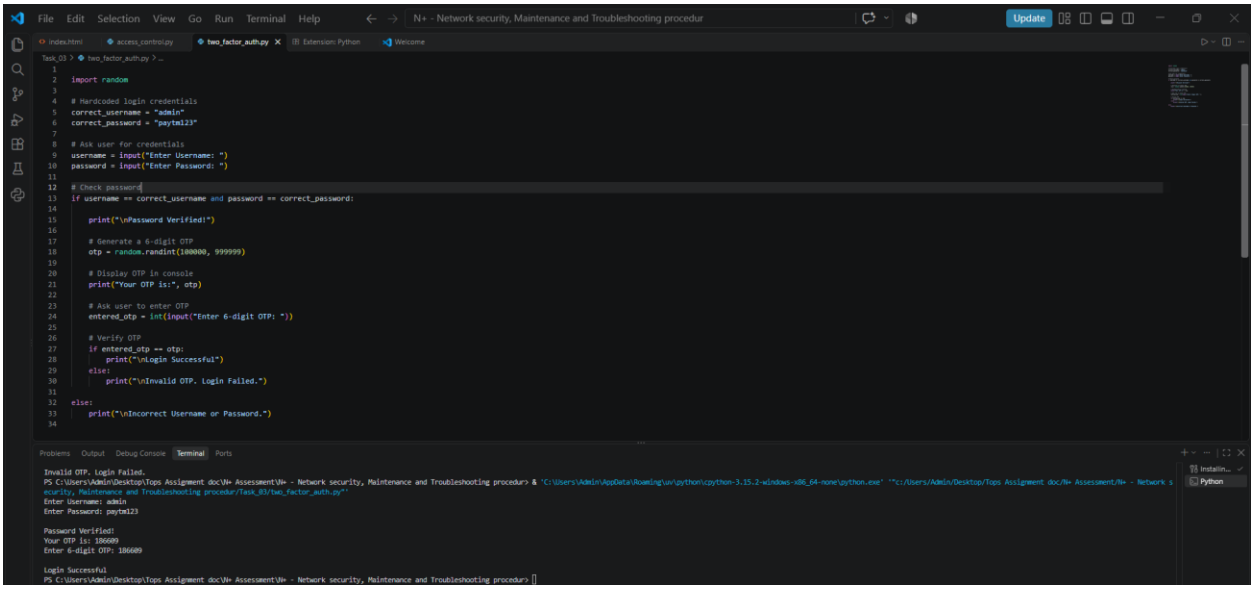
**Task 3 : Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.**

**Description:**
The program first verifies the username and password. If the credentials are correct, it generates a random 6-digit OTP and displays it in the console. The user must enter the correct OTP to complete the login process.

**Authentication Steps:**

1. Enter username and password.
2. Verify login credentials.
3. Generate a 6-digit OTP.
4. Enter the OTP.
5. Display Login Successful if the OTP matches.

**Output:**
The program successfully verifies the password and OTP before granting access.



**Task 4 : Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.**

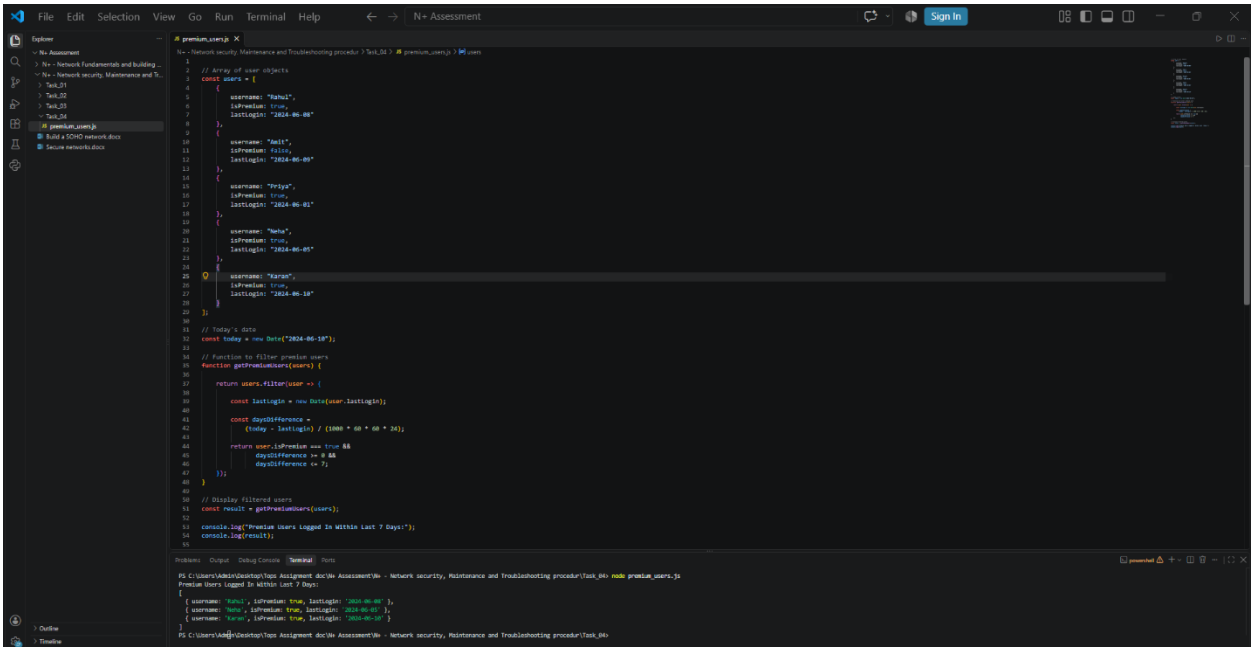
**Description:**
An array of user objects is created with username, isPremium and lastLogin properties. The filter() method is used to select only premium users whose last login date falls within the last 7 days.

**Today's Date:** 2024-06-10

**Conditions:**

1. isPremium must be true.
2. Last login must be within the last 7 days.
3. Users with future login dates are excluded.

**Output:**
The program successfully displays only premium users who logged in within the last 7 days.



**Task 5 : Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.**

To identify five common access control mistakes made by developers in real-world applications like Instagram and Flipkart and understand their prevention methods.

**1. Broken Access Control**

**Mistake:** Users can access pages or information that they are not authorized to view.

**Prevention:** Always verify user permissions before allowing access to any resource.

**2. Weak Password Protection**

**Mistake:** Developers allow weak passwords that can be easily guessed or cracked.

**Prevention:** Enforce strong password policies and securely hash passwords using algorithms like Argon2 or bcrypt.

**3. Improper Role-Based Access Control**

**Mistake:** Users receive unnecessary permissions, such as customers accessing admin features.

**Prevention:** Assign permissions according to user roles and follow the principle of least privilege.

**4. Insecure Session Management**

**Mistake:** Applications fail to properly protect user sessions, allowing attackers to misuse session IDs.

**Prevention:** Use secure session cookies, regenerate session IDs after login, and implement session expiration.

**5. Insecure Direct Object Reference (IDOR)**

**Mistake:** Users can access another user's private data by changing an ID in the URL, such as an order ID or profile ID.

**Prevention:** Verify resource ownership and authorization on the server side before displaying or modifying data.